EthernaLotto

Author: James Farrell, <farrelljames558@gmail.com>
Status: Draft

Abstract

We hereby provide the design specification of **EthernaLotto**, a lottery game based on the Ethereum Virtual Machine ("EVM" in the following).

The game will be managed as a Decentralized Autonomous Organization ("DAO" in the following).

For scalability reasons, the game will be deployed to the Polygon PoS chain (aka Matic) at this time. In the future we will probably migrate to Ethereum 2.

Motivations

Lottery games have ancient origins tracing back to the Chinese Han Dynasty, between years 205 and 187 BC. The core idea of the game is that every player pays a small amount and a randomly chosen lucky winner gets the money from all contributions.

Unfortunately, classic operation of such games requires a central running authority and subjects them to serious issues:

- **Censorship** - some countries make some or all lotteries illegal.
- **Locality** - lotteries are usually State-managed and subject to local regulations, therefore it is physically impossible to extend them across borders and turn them global.
- **Excessive taxation** - the actual proceeds from a win are usually significantly lower than the advertised jackpot.
- **Unfair taxation** - taxation may change over time, making the process unfair to all players who bought a ticket *before* a change if the draw happens *after* the change.
- **Obscure or unfair prize allocation** - in a typical lottery game a share of the jackpot is assigned to every winning category, but the way these shares are calculated is almost never explained and is often unfair.
- **Outright scams** - classic operation provides little to no guarantees about the fairness of the drawing process, as well as the actual value of the ticket sales. The drawn numbers may be biased to favor specific players, and / or the lottery runner may advertise a jackpot that is significantly lower (but still attractive enough) than the ticket sales. It is certainly suspicious that many lotteries constantly advertise perfectly round-numbered jackpots like $1M.

Case in point, Hot Lotto suffered one of the most famous lottery fraud scandals (not necessarily the worst). Other frauds that might have occurred may never be uncovered.

We want to overcome all the above problems by implementing a new lottery game based on a decentralized, trustless, censorship-resistant, and cryptographically verified platform: the EVM.

Game Description

Players will play the game by buying one or more tickets using the native currency of the Polygon PoS chain, MATIC. 10% of the earnings from ticket sales will be transferred to the DAO running the game and distributed among shareholders, while the remaining 90% will be used as a jackpot.

When buying a ticket, a player will choose 6 or more different numbers between 1 and 90 inclusive, and exactly 6 different numbers in the same range will be drawn randomly every week by the system. Tickets matching 2 or more of the drawn numbers will be given a prize.

Neither the order of the numbers played in a ticket nor that of the 6 drawn numbers matters.

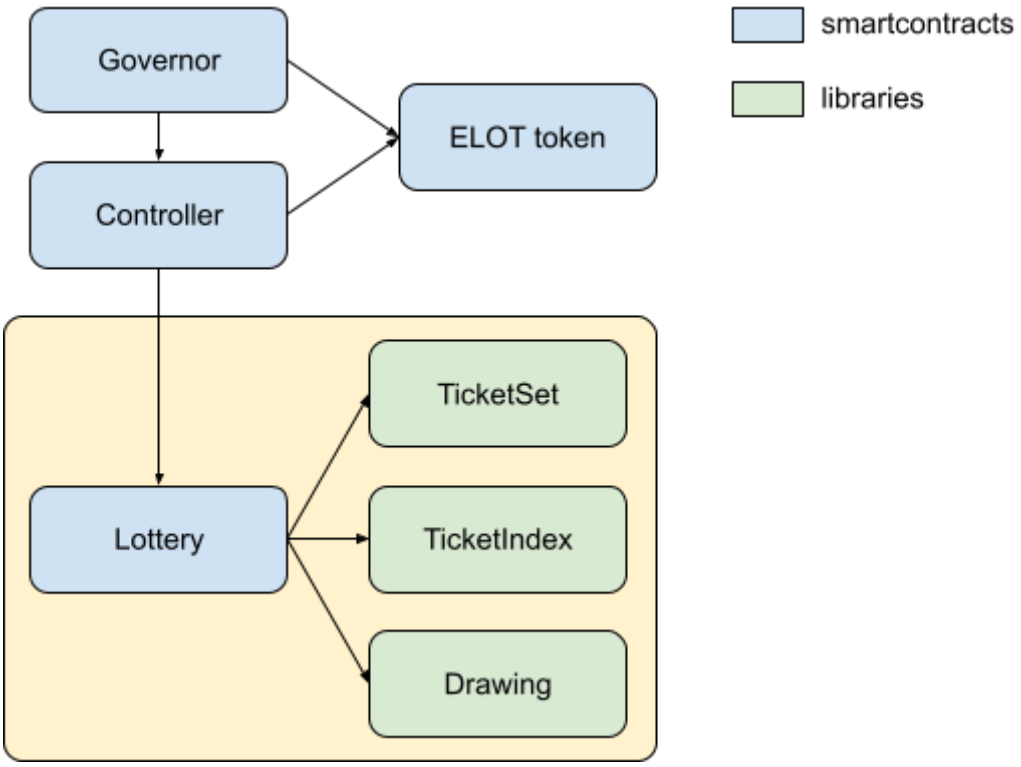
Tickets with more than 6 numbers are treated exactly as if the player had bought many tickets for all possible 6-combinations of those numbers. For example, the price of a 10-number ticket is $\binom{10}{6}$ times the price of a 6-number ticket.[1]

Players are identified by their Ethereum addresses and will be able to withdraw any prizes won by their tickets after each draw. If a player buys more than one ticket, any prizes from all of their tickets will add up.

The drawing process will use ChainLink's Verifiable Random Function, or VRF for short, which guarantees fairness and prevents cheating.

Architecture

The following is a high-level view of the components that make up EthernaLotto:



[1] https://en.wikipedia.org/wiki/Binomial_coefficient

Each block in the diagram represents a separate smartcontract or library. The implementation of all contracts makes extensive use of OpenZeppelin smartcontracts v4 and can be found in this GitHub repository.

The **Lottery** smartcontract implements the core logic of the lottery game. Since the algorithms turned out to be quite complex, some of the functionality has been isolated into the **TicketSet**, **TicketIndex**, and **Drawing** libraries so as to keep the contract size within the 24 KiB limit enforced by the EVM.

The whole **Lottery**+**TicketSet**+**TicketIndex**+**Drawing** group can be thought of as a single black box providing two high-level features: *buying tickets* and *drawing numbers*. The former is public (anyone can buy a ticket), while the latter can be run only by an unspecified *owner* of the contract, as per OpenZeppelin's Ownable contract.

The **Controller** contract is an OpenZeppelin `TimelockController` with some additional functionality. The **Lottery** contract group described above does not implement any mechanism to control the frequency of the draws: a non-standard deployment of those contracts without any Controller may theoretically be owned by a private address triggering draws at will, e.g. every day or at variable frequency. The fairness of the official deployment is guaranteed by making the Controller the only owner of the Lottery group, so that all draws are triggered in a standard way. Namely, the Controller allows drawing no more than once every 6 days and 23 hours.

The **Governor** contract is a regular OpenZeppelin Governor with specific OpenZeppelin extensions.

The Lottery contract is the only upgradeable one, while all others have been kept non-upgradeable for simplicity. If the DAO decides to perform an upgrade to its own structure it will need to deploy a new Governor+Controller pair and set the new Controller as the owner of the Lottery.

The next two sections describe the algorithms used to index the tickets and identify the winners as efficiently as possible. These algorithms have been optimized in several ways to reduce both storage costs and computational costs.

Storage Structure

The Lottery contract keeps tracks of the sold tickets and their respective buyers.

Tickets are identified by a 64-bit incremental ID starting at 0.

Tickets are stored in a two-dimensional array called `ticketsByNumber`, which indexes them by their numbers:

```
uint64[][90] public ticketsByNumber;
```

The primary index of the array ranges from 0 to 89, matching the range of the playable numbers (offset by 1 because array indices are zero-based). Each element of the 90-element primary array is in turn an array containing the set of tickets that played that number.

For example, the expression `ticketsByNumber[22]` yields the set of all tickets that played the number 23.

Note that a ticket can and will appear more than once in the `ticketsByNumber` data structure: in fact it will appear at least 6 times, in 6 different sets. It cannot appear more than once in the same set because the played numbers must not have repetitions.

The `TicketSet` library is used for `uint64[]` arrays and provides methods to run various algorithms on ticket sets. The `TicketIndex` library is used for the whole `ticketsByNumber` data structure and provides a core piece of the drawing algorithm described in Round Closure; it makes use of the lower-level algorithms from `TicketSet`.

Ticket buyers (i.e. players) are stored in another array called `playersByTicket`, which indexes them by ticket:

```
address payable[] public playersByTicket;
```

Since ticket IDs are incremental and start at 0, they can be used as indices in an array. The index of the `playersByTicket` array is the ticket number, while the corresponding element is the address of the player who bought it. An address will show up multiple times if a player bought many tickets in the same round.

As an example, the `playersByTicket[12]` expression yields the address of the player who bought the 13th ticket in the current round.

An important invariant is that the insertion of a ticket in a set always happens by appending it at the end. Since ticket IDs are incremental it follows that all ticket sets are ordered arrays. The algorithms involved in the drawing process actively exploit this fact.

Aside from the two data structures described above, the Lottery contract contains a few other fields to describe the status of the game. At any given time the game can be in one of four possible states:

- OPEN,
- DRAWING,
- DRAWN,
- CLOSING.

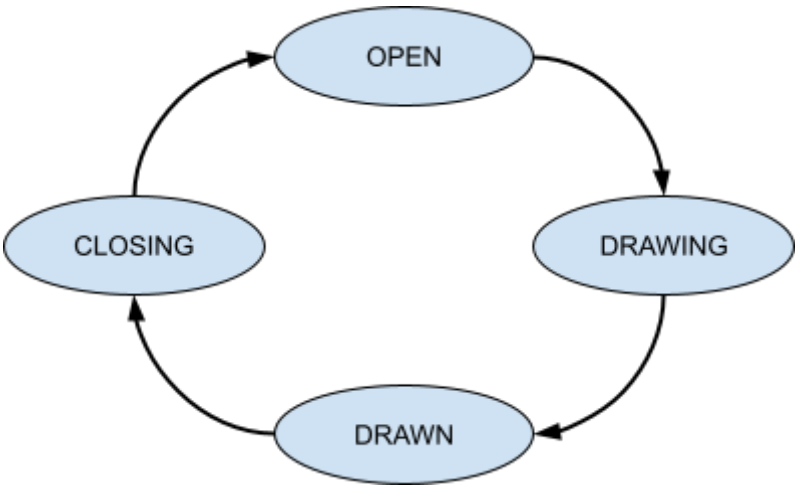
The OPEN state indicates that the game is selling tickets.

The DRAWING state indicates that ticket sales are closed and the ChainLink VRF has been triggered.

The DRAWN state indicates that the ChainLink VRF Coordinator has returned the randomness source and the game is ready to identify winners. See the Drawing Process section for more details.

The CLOSING state indicates that winning tickets have been identified and a separate call can be triggered to attribute prizes to the respective players.

The allowed state transitions are:



In theory the DRAWING, DRAWN, and CLOSING state could be merged into one if all the work for identifying winners and attributing prizes was done in the VRF callback, but in practice these two operations take up a large amount of gas and we prefer to manage them separately.

Drawing Process

Overview

The VRF callback uses the provided 256-bit randomness to pick 6 different random numbers, which is achieved by initializing an array with the first 90 naturals, scrambling the first 6 slots using the Fisher-Yates shuffle, and returning the first 6 elements.

The drawing algorithm (implemented in the `Drawing` library) follows:

```
function getRandomNumbersWithoutRepetitions(uint256 randomness)
    public pure returns (uint8[6] memory numbers)
{
  uint8[90] memory source;
  for (uint8 i = 1; i <= 90; i++) {
    source[i - 1] = i;
  }
  for (uint i = 0; i < 6; i++) {
    uint j = i + randomness % (90 - i);
    randomness /= 90;
    numbers[i] = source[j];
    source[j] = source[i];
  }
}
```

Once the 6 numbers are known the lottery progresses into the DRAWN state and allows anyone to trigger **round closure**, which consists of the following steps:

1. identifying all tickets that matched two or more of the drawn numbers,
2. attributing the respective prizes so that they are available to the players for withdrawal, and
3. resetting the state of the game.

Steps #1 and #2 are the most complex part of the system, and are described in the rest of this section.

Round Closure

The `ticketsByNumber` data structure described in the Storage Structure section allows retrieving the set of tickets that played a specific number efficiently. Having that, we can enumerate all possible ways to choose:

- 6 of the drawn numbers,
- 5 of the drawn numbers,
- 4 of the drawn numbers,
- 3 of the drawn numbers,
- and 2 of the drawn numbers.

This way we are enumerating all possible ways to choose 2 or more numbers from the 6 drawn ones. For every enumerated way we get a combination of numbers, and for every combination we need to:

- look up the tickets that played the 1st number,
- look up the tickets that played the 2nd number,
- …

and **intersect all looked up sets**. The resulting intersection is the list of tickets winning that combination, which may be empty.

We will now describe an algorithm called `findWinningTickets` which takes the `ticketsByNumber` data structure defined in Storage Structure and the 6 drawn numbers as input, and returns a matrix containing the IDs of the winning tickets in each category as output. The output matrix is an array of 5 arrays, one for each win category, and each secondary array is an ordered list of ticket IDs.

The signature of this algorithm is:

```
function findWinningTickets(
    uint64[][90] storage ticketsByNumber,
    uint8[6] storage numbers
)
    public
    view
    returns (uint64[][5] memory winners)
{
  // …
}
```

Note that the returned tickets are **not** weighted at this stage – the caller is in charge of weighing them as described in Weighting.

The following nested cycle structure is used to scan all possible combinations (aka the "parts") of the drawn numbers:

```
for (uint i0 = 0; i0 < 6; i0++) {
  for (uint i1 = i0 + 1; i1 < 6; i1++) {
    for (uint i2 = i1 + 1; i2 < 6; i2++) {
```

```
        for (uint i3 = i2 + 1; i3 < 6; i3++) {
          for (uint i4 = i3 + 1; i4 < 6; i4++) {
            for (uint i5 = i4 + 1; i5 < 6; i5++) {
              // ...
            }
          }
        }
      }
    }
  }
```

In fact, adding the following calls to the [Hardhat console]():

```
  for (uint i0 = 0; i0 < 6; i0++) {
    for (uint i1 = i0 + 1; i1 < 6; i1++) {
      for (uint i2 = i1 + 1; i2 < 6; i2++) {
        for (uint i3 = i2 + 1; i3 < 6; i3++) {
          for (uint i4 = i3 + 1; i4 < 6; i4++) {
            for (uint i5 = i4 + 1; i5 < 6; i5++) {
              // ...
              console.log(i0, i1, i2, i3, i4, i5);
            }
            console.log(i0, i1, i2, i3, i4);
          }
          console.log(i0, i1, i2, i3);
        }
        console.log(i0, i1, i2);
      }
      console.log(i0, i1);
    }
    console.log(i0);
  }
```

yields something like:

```
0 1 2 3 4 5
0 1 2 3 4
0 1 2 3 5
0 1 2 3
0 1 2 4 5
0 1 2 4
0 1 2 5
0 1 2
0 1 3 4 5
0 1 3 4
...
3 5
3
4 5
4
```

```
5
```

For a total 63 rows, or 64 if we include the empty set. The combinations of cardinality 0 and 1, namely {}, {0}, {1}, {2}, {3}, {4}, and {5}, are not relevant for the algorithm.

The `ticketsByNumber` data structure provides all the tickets containing a given number. The **TicketSet** library provides an intersection algorithm allowing us to progressively identify the tickets which played a given combination. Combining the two we get:

```
for (uint i0 = 0; i0 < 6; i0++) {
  uint64[] memory tickets0 = ticketsByNumber[numbers[i0]];
  for (uint i1 = i0 + 1; i1 < 6; i1++) {
    uint64[] memory tickets1 = tickets0.intersect(
        ticketsByNumber[numbers[i1]]);
    for (uint i2 = i1 + 1; i2 < 6; i2++) {
      uint64[] memory tickets2 = tickets1.intersect(
          ticketsByNumber[numbers[i2]]);
      for (uint i3 = i2 + 1; i3 < 6; i3++) {
        uint64[] memory tickets3 = tickets2.intersect(
            ticketsByNumber[numbers[i3]]);
        for (uint i4 = i3 + 1; i4 < 6; i4++) {
          uint64[] memory tickets4 = tickets3.intersect(
              ticketsByNumber[numbers[i4]]);
          for (uint i5 = i4 + 1; i5 < 6; i5++) {
            uint64[] memory tickets5 = tickets4.intersect(
                ticketsByNumber[numbers[i5]]);
          }
        }
      }
    }
  }
}
```

`tickets0` contains the tickets matching only 1 number, so it will be discarded. On the other hand, variables `tickets1` through `tickets5` contain actual winners, so they need to be returned in the output.

Let us now initialize and fill in the output data structure:

```
winners = [
  new uint64[](0),  // tickets matching 2 numbers
  new uint64[](0),  // tickets matching 3 numbers
  new uint64[](0),  // tickets matching 4 numbers
  new uint64[](0),  // tickets matching 5 numbers
  new uint64[](0)   // tickets matching 6 numbers
];
for (uint i0 = 0; i0 < 6; i0++) {
  uint64[] memory tickets0 = ticketsByNumber[numbers[i0]];
  for (uint i1 = i0 + 1; i1 < 6; i1++) {
    uint64[] memory tickets1 = tickets0.intersect(
        ticketsByNumber[numbers[i1]]);
```

```
    for (uint i2 = i1 + 1; i2 < 6; i2++) {
      uint64[] memory tickets2 = tickets1.intersect(
          ticketsByNumber[numbers[i2]]);
      for (uint i3 = i2 + 1; i3 < 6; i3++) {
        uint64[] memory tickets3 = tickets2.intersect(
            ticketsByNumber[numbers[i3]]);
        for (uint i4 = i3 + 1; i4 < 6; i4++) {
          uint64[] memory tickets4 = tickets3.intersect(
              ticketsByNumber[numbers[i4]]);
          for (uint i5 = i4 + 1; i5 < 6; i5++) {
            uint64[] memory tickets5 = tickets4.intersect(
                ticketsByNumber[numbers[i5]]);
            winners[4] = winners[4].union(tickets5);
          }
          winners[3] = winners[3].union(tickets4);
        }
        winners[2] = winners[2].union(tickets3);
      }
      winners[1] = winners[1].union(tickets2);
    }
    winners[0] = winners[0].union(tickets1);
  }
}
```

The algorithm written so far does not respect the property that a ticket winning in one category must not win in all the categories below. For example, a ticket with 4 matches will make its way to `tickets3`, but as the algorithm unwinds back to the top it will still be present in `tickets2` and `tickets1`, so it will be incorrectly added to `winners[1]` and `winners[0]`.

There is no easy way to prevent this, because the highest winning category of a given ticket will not necessarily be detected on the first branch of the execution tree where that ticket is processed. Therefore we need to clean up the output data structure at the end with the following addition:

```
for (uint j = 0; j < winners.length - 1; j++) {
  for (uint k = j + 1; k < winners.length; k++) {
    winners[j] = winners[j].subtract(winners[k]);
  }
}
```

Last but not least, we can implement a few optimizations.

First, if at some point during the execution of the 6 stages we end up with an empty ticket set, we can bail out early:

```
winners = [
  new uint64[](0),  // tickets matching exactly 2 numbers
  new uint64[](0),  // tickets matching exactly 3 numbers
  new uint64[](0),  // tickets matching exactly 4 numbers
  new uint64[](0),  // tickets matching exactly 5 numbers
  new uint64[](0)   // tickets matching exactly 6 numbers
```

```
];
for (uint i0 = 0; i0 < 6; i0++) {
  uint64[] memory tickets0 = ticketsByNumber[numbers[i0]];
  if (tickets0.length > 0) {
    for (uint i1 = i0 + 1; i1 < 6; i1++) {
      uint64[] memory tickets1 = tickets0.intersect(
          ticketsByNumber[numbers[i1]]);
      if (tickets1.length > 0) {
        for (uint i2 = i1 + 1; i2 < 6; i2++) {
          uint64[] memory tickets2 = tickets1.intersect(
              ticketsByNumber[numbers[i2]]);
          if (tickets2.length > 0) {
            for (uint i3 = i2 + 1; i3 < 6; i3++) {
              uint64[] memory tickets3 = tickets2.intersect(
                  ticketsByNumber[numbers[i3]]);
              if (tickets3.length > 0) {
                for (uint i4 = i3 + 1; i4 < 6; i4++) {
                  uint64[] memory tickets4 = tickets3.intersect(
                      ticketsByNumber[numbers[i4]]);
                  if (tickets4.length > 0) {
                    for (uint i5 = i4 + 1; i5 < 6; i5++) {
                      uint64[] memory tickets5 = tickets4.intersect(
                          ticketsByNumber[numbers[i5]]);
                      if (tickets5.length > 0) {
                        winners[4] = winners[4].union(tickets5);
                      }
                    }
                    winners[3] = winners[3].union(tickets4);
                  }
                }
                winners[2] = winners[2].union(tickets3);
              }
            }
            winners[1] = winners[1].union(tickets2);
          }
        }
        winners[0] = winners[0].union(tickets1);
      }
    }
  }
}
for (uint j = 0; j < winners.length - 1; j++) {
  for (uint k = j + 1; k < winners.length; k++) {
    winners[j] = winners[j].subtract(winners[k]);
  }
}
```

Secondly, we can increase our chances of emptying a ticket set by subtracting the tickets of the next set early (if they are found in the next set it means they have won in a higher category, so they are not supposed to be in the current set).

```
winners = [
  new uint64[](0),  // tickets matching exactly 2 numbers
  new uint64[](0),  // tickets matching exactly 3 numbers
  new uint64[](0),  // tickets matching exactly 4 numbers
  new uint64[](0),  // tickets matching exactly 5 numbers
  new uint64[](0)   // tickets matching exactly 6 numbers
];
for (uint i0 = 0; i0 < 6; i0++) {
  uint64[] memory tickets0 = ticketsByNumber[numbers[i0]];
  if (tickets0.length > 0) {
    for (uint i1 = i0 + 1; i1 < 6; i1++) {
      uint64[] memory tickets1 = tickets0.intersect(
          ticketsByNumber[numbers[i1]]);
      if (tickets1.length > 0) {
        tickets0 = tickets0.subtract(tickets1);
        for (uint i2 = i1 + 1; i2 < 6; i2++) {
          uint64[] memory tickets2 = tickets1.intersect(
              ticketsByNumber[numbers[i2]]);
          if (tickets2.length > 0) {
            tickets1 = tickets1.subtract(tickets2);
            for (uint i3 = i2 + 1; i3 < 6; i3++) {
              uint64[] memory tickets3 = tickets2.intersect(
                  ticketsByNumber[numbers[i3]]);
              if (tickets3.length > 0) {
                tickets2 = tickets2.subtract(tickets3);
                for (uint i4 = i3 + 1; i4 < 6; i4++) {
                  uint64[] memory tickets4 = tickets3.intersect(
                      ticketsByNumber[numbers[i4]]);
                  if (tickets4.length > 0) {
                    tickets3 = tickets3.subtract(tickets4);
                    for (uint i5 = i4 + 1; i5 < 6; i5++) {
                      uint64[] memory tickets5 = tickets4.intersect(
                          ticketsByNumber[numbers[i5]]);
                      if (tickets5.length > 0) {
                        tickets4 = tickets4.subtract(tickets5);
                        winners[4] = winners[4].union(tickets5);
                      }
                    }
                    winners[3] = winners[3].union(tickets4);
                  }
                }
                winners[2] = winners[2].union(tickets3);
              }
            }
            winners[1] = winners[1].union(tickets2);
          }
        }
        winners[0] = winners[0].union(tickets1);
      }
    }
```

```
    }
  }
  for (uint j = 0; j < winners.length - 1; j++) {
    for (uint k = j + 1; k < winners.length; k++) {
      winners[j] = winners[j].subtract(winners[k]);
    }
  }
```

Now the generic stage of the algorithm turns out as follows:

```
  for (uint i2 = i1 + 1; i2 < 6; i2++) {
    uint64[] memory tickets2 = tickets1.intersect(
        ticketsByNumber[numbers[i2]]);
    if (tickets2.length > 0) {
      tickets1 = tickets1.subtract(tickets2);

      ...

      winners[1] = winners[1].union(tickets2);
    }
  }
```

It is still advantageous to subtract `tickets1` from `tickets0` even though `tickets0` does not participate in the output because doing so increases the likelihood of emptying all sets and stepping directly to the next `i0` index.

The last optimization must actually be implemented by the *caller* of `findWinningTickets`. In order to further increase the chance of reducing or emptying a set early, we need to process the drawn numbers in order from the least frequently played to the most frequently played. So the caller needs to sort them by `ticketsByNumber[...].length`.

Since we are sorting only 6 elements, implementing efficient $O(N \cdot log(N))$ sorting was an overkill and probably even counterproductive, so we simply implemented the following quadratic time selection sort (in the `Drawing` library):

```
  function sortNumbersByTicketCount(
      uint64[][90] storage ticketsByNumber,
      uint8[6] memory number
  )
      public
      view
      returns (uint8[6] memory)
  {
    for (uint i = 0; i < numbers.length - 1; i++) {
      uint j = i;
      for (uint k = j + 1; k < numbers.length; k++) {
        if (ticketsByNumber[numbers[k]].length <
            ticketsByNumber[numbers[j]].length)
        {
          j = k;
        }
```

```
      }
      if (j != i) {
        uint8 t = numbers[i];
        numbers[i] = numbers[j];
        numbers[j] = t;
      }
    }
    return numbers;
  }
}
```

Intersections

Thanks to the fact that ticket sets are ordered, intersections can often be calculated in sub-linear time by skipping leading ranges. For example, if we are intersecting sets A and B, with `A[0] = 12345` and `B[0] < 12345`, we can skip the leading range of B in logarithmic time until we find an element that is greater than or equal to 12345. In the best case scenario where `A[0] > B[B.length - 1]`, A and B do not have any elements in common and the entire sets will be skipped in logarithmic time.

TODO

Prize Distribution

Overview

Five win categories are defined:

1. tickets matching exactly 2 of the drawn numbers,
2. tickets matching exactly 3 of the drawn numbers,
3. tickets matching exactly 4 of the drawn numbers,
4. tickets matching exactly 5 of the drawn numbers,
5. and tickets matching all 6 drawn numbers.

The jackpot is divided in 6 parts, one part for each category and then an extra part to fund the jackpot of the next round in case all categories have at least one winner.

Note that a winning ticket is awarded a prize only in the highest winning category. For example, if a ticket matches 4 numbers it will not be assigned any prizes from the 3-match and 2-match categories. That is also why a given ticket can win only once: if, for example, a ticket matches two different 3-combinations of the drawn set, and the union of those combinations is a set of 4 numbers, then the ticket is awarded a single prize from the 4-match category, not 2 from the 3-match category.

If a category has no winners in a given round, the corresponding share of the jackpot will be kept for the next round.

Weighting

As mentioned in the Game Description section, EthernaLotto treats tickets with more than 6 numbers exactly as if they were many tickets for all possible 6-combinations of the chosen numbers. This means that tickets with a different number of played numbers deserve a different prize even if they win in the same category. For instance, if we have these two tickets:

1, 2, 3, 4, 5, 6, 7, 8

1, 2, 3, 4, 5, 6

and the drawn numbers are:

1, 2, 3, 4, 89, 90

then both tickets win in the 4-match category, but the first ticket is equivalent to 6 different 6-number tickets that include 1, 2, 3, and 4, while the second ticket is equivalent to only 1 of them. Therefore the first ticket deserves a prize that is 6 times greater than the second.

The exact formula to weigh each ticket winning in the generic k-match category is $\binom{n-k}{6-k}$, with n being the number of numbers in the ticket and k being the number of matches.

Precalculating these binomial coefficients requires a table of all possible (n, k) pairs, with some arbitrary limits on n (k always ranges from 2 to 6 inclusive). Rather than hard-coding this table we decided to implement proper calculation of the binomial coefficient, which we also reused to calculate the prices of the tickets. We used the following algorithm:

```
function _choose(uint n, uint k) private pure returns (uint) {
   require(n >= k, 'internal error, invalid binomial coefficient');
   if (k == 0) {
     return 1;
   } else if (k * 2 > n) {
     return _choose(n, n - k);
   } else {
     return n * _choose(n - 1, k - 1) / k;
   }
 }
```

That is based on the following:

$$\binom{n}{k} = \frac{n}{k} \cdot \binom{n-1}{k-1}$$

Easily proven as follows:

$$\frac{n!}{k!(n-k)!} = \frac{n}{k} \cdot \frac{(n-1)!}{(k-1)!(n-k)!}$$

Jackpot Subdivision

The 6 shares of the jackpot are hard-coded and have been calculated as follows: 10% is always kept for the next round, while each win category is assigned an 18% share. The shares for the various win categories all have the same size because, in order to award the fairest possible prizes, the share for a given category must be inversely proportional to the probability of winning and directly proportional to the expected number of winners. In other words:

- it is fair to allocate a larger share for the winners of a *higher* category because they have a *lower* probability of winning; and
- it is fair to allocate a larger share for the winners of a *lower* category because we expect *more* winners.

So we have two conflicting criteria that balance each other out, as we show formally in the following.

For a given category k we can call the probability of winning in that category p_k, and the expected number of winners ξ_k. We can call the total number of played 6-combinations in a round n. ξ_k is a binomial variable, so its expected value is np_k. The share of the jackpot for category k can be measured as:

$$S_k = \frac{E(\xi_k)}{p_k} = \frac{np_k}{p_k} = n$$

This measure does not depend on p_k or any other k-dependent parameters, so **all shares must have the same size**.

DAO and Governance

As mentioned earlier, the game will be managed as a *Decentralized Autonomous Organization*, or DAO for short.

Participation in the DAO will be determined by a newly minted ERC20 token (trading symbol: `ELOT`). The voting power of each participating address will be proportional to the amount of owned tokens. This is very standard practice in the Ethereum ecosystem.

Voting and upgrades are all performed based on the standard models provided by OpenZeppelin, so we will not cover them in this paper.

TODO

Deployment Sequence

TODO

Alternatives Considered

Choosing the winning addresses randomly

Rather than creating a number-based game and implementing the complex algorithms described in the Drawing Process section we could have simply used ChainLink's randomness to choose one player (or a few ones) randomly.

This is fundamentally different from a traditional number-based game because it does not require any input on the player's side. It also rules out the possibility that many players win in different categories, or that no one wins at all.

Guessing lucky numbers is a much more interesting game, and the various categories only add to the thrill!

Alternative indexing algorithm

The algorithms described in the Drawing Process section take up a lot of gas, and in order for the game to be sustainable the cost of that gas will always have to be funded by (a small percentage of) the ticket sales.

In order to make sure that the bulk of the gas cost is always charged on the players, we initially designed and implemented a completely different data structure to index the tickets, and a completely different drawing process. We hereby describe this alternative model.

Let us define a bijection from the first 90 natural numbers 0, 1, 2, … to the first 90 primes 2, 3, 5, …

We have:

$0 \leftrightarrow 2$
$1 \leftrightarrow 3$
$2 \leftrightarrow 5$
$3 \leftrightarrow 7$
$4 \leftrightarrow 11$
$5 \leftrightarrow 13$
$6 \leftrightarrow 17$
…
$87 \leftrightarrow 457$
$88 \leftrightarrow 461$
$89 \leftrightarrow 463$

As usual, the first 90 naturals correspond to the numbers that can be played in EthernaLotto, offset by 1 so that they are zero-based.

Let us call the above table p so that p_i is the i-th prime.

A ticket playing 6 numbers a_0, a_1, a_2, a_3, a_4, and a_5, **independently of their order**, can be uniquely identified by the following product:

$$h = p_{a_0} \cdot p_{a_1} \cdot p_{a_2} \cdot p_{a_3} \cdot p_{a_4} \cdot p_{a_5}$$

So we can index it in a Solidity `mapping` where a key is the product h and the corresponding value is an array of addresses corresponding to the set of players who played those numbers.

The drawing process then becomes much simpler because we can just calculate the product of the 6 drawn numbers and the list of winners is found by looking it up in the mapping.

This provides the general idea, while the actual algorithm was more complex because it had to account for all possible ways to choose 2 of the played numbers, 3 of them, 4 of them, etc. and had to index the ticket in all corresponding slots of the mapping. A ticket playing 6 numbers would then be stored in:

$$\binom{6}{2} + \binom{6}{3} + \binom{6}{4} + \binom{6}{5} + \binom{6}{6} = 15 + 20 + 15 + 6 + 1$$

… = 57 storage slots.

Note that we did not need to account for $\binom{6}{1}$ and store the ticket in 63 slots because tickets matching only 1 number do not win any prize.

This model was discarded because in our tests it used way more gas than the current one overall, and posed a risk of inflating the ticket prices to unacceptable levels even on the Polygon PoS chain. Through this experience we learned that storage costs are much more critical than computational costs in the EVM.

Sharded drawings

Since the drawing process is gas-heavy, we have considered sharding it as follows.

Enumerating all possible subsets of 6 elements (the 6 drawn numbers) is equivalent to counting from 0 to 63 in binary: it takes exactly 6 bits and the i-th bit can be interpreted as a boolean flag indicating whether the i-th drawn number is included in the current subset. Numbers with less than 2 bits set, namely 0 and the powers of 2, can be discarded.

This way the process of identifying the winning tickets can be divided in $64 - 7 = 57$ smaller parts, allowing for better manageability. For example, if the transaction for the 50th shard fails we would not have to repeat all previous 49 shards.

This approach had to be discarded because overall it uses substantially more gas than the `findWinningTickets` algorithm described in Round Closure.

TODO